

Лабораторная работа № 13

**Программирование в командном процессоре ОС UNIX. Ветвления и
циклы**

Жукова София Викторовна

Содержание

1	Цель работы	5
2	Задание	6
3	Выполнение лабораторной работы	7
4	Выводы	19

Список иллюстраций

3.1	Создаем файлы	7
3.2	Пишем текст	8
3.3	Пишем код	9
3.4	Проверяем	10
3.5	Создаем файлы	10
3.6	Пишем код	11
3.7	Пишем код	12
3.8	Проверяем	12
3.9	Создаем файл	12
3.10	Пишем код	13
3.11	Запускаем	13
3.12	Проверяем	14
3.13	Запускаем	14
3.14	Проверяем	15
3.15	Создаем файл	15
3.16	Пишем код	16
3.17	Запускаем	17
3.18	Проверяем	17
3.19	Распаковываем	18
3.20	Проверяем	18

Список таблиц

1 Цель работы

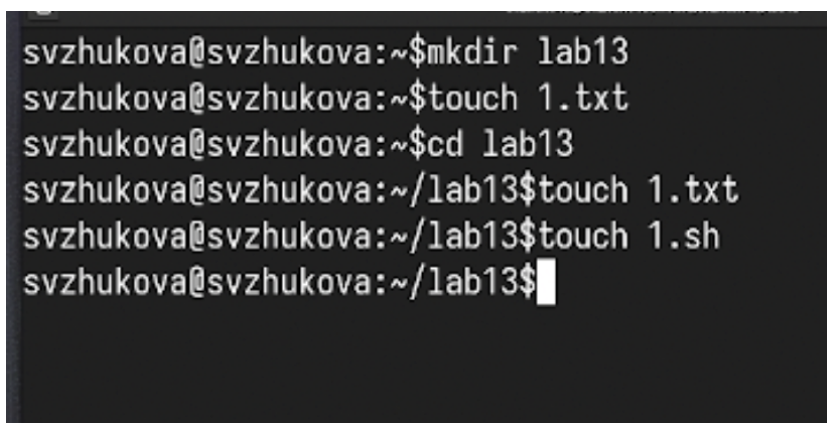
Изучить основы программирования в оболочке ОС UNIX. Научится писать более сложные командные файлы с использованием логических управляющих конструкций и циклов.

2 Задание

1. Используя команды `getopts` `grep`, написать командный файл, который анализирует командную строку с ключами: `-iinputfile` — прочитать данные из указанного файла; `-ooutputfile` — вывести данные в указанный файл; `-r` — шаблон — указать шаблон для поиска; `-C` — различать большие и малые буквы; `-n` — выдавать номера строк. а затем ищет в указанном файле нужные строки, определяемые ключом `-r`.
2. Написать на языке Си программу, которая вводит число и определяет, является ли оно больше нуля, меньше нуля или равно нулю. Затем программа завершается с помощью функции `exit(n)`, передавая информацию в о коде завершения в оболочку. Командный файл должен вызывать эту программу и, проанализировав с помощью команды `$?`, выдать сообщение о том, какое число было введено.
3. Написать командный файл, создающий указанное число файлов, пронумерованных последовательно от 1 до n (например 1.tmp, 2.tmp, 3.tmp, 4.tmp и т.д.). Число файлов, которые необходимо создать, передаётся в аргументы командной строки. Этот же командный файл должен уметь удалять все созданные им файлы (если они существуют).
4. Написать командный файл, который с помощью команды `tag` запаковывает в архив все файлы в указанной директории. Модифицировать его так, чтобы запаковывались только те файлы, которые были изменены менее недели тому назад (использовать команду `find`)

3 Выполнение лабораторной работы

Создадим файл с расширением sh и текстовый файл (рис. 3.1).

A terminal window with a dark background and light-colored text. It shows a series of commands being entered and executed. The prompt is 'svzhukova@svzhukova:~\$'. The commands are: 'mkdir lab13', 'touch 1.txt', 'cd lab13', 'touch 1.txt', 'touch 1.sh', and finally 'svzhukova@svzhukova:~/lab13\$' with a cursor at the end.

```
svzhukova@svzhukova:~$mkdir lab13
svzhukova@svzhukova:~$touch 1.txt
svzhukova@svzhukova:~$cd lab13
svzhukova@svzhukova:~/lab13$touch 1.txt
svzhukova@svzhukova:~/lab13$touch 1.sh
svzhukova@svzhukova:~/lab13$
```

Рис. 3.1: Создаем файлы

Заполняем текстовый файл (рис. 3.2).

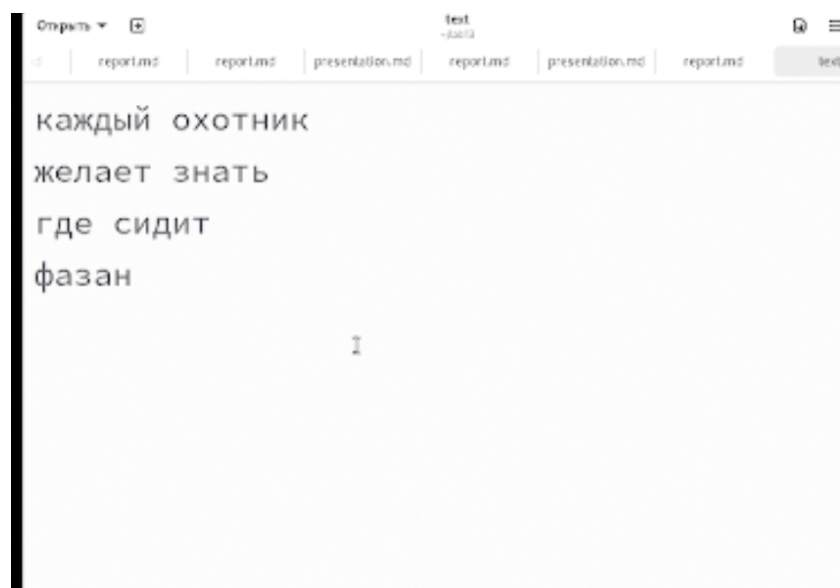


Рис. 3.2: Пишем текст

Пишем код, который анализирует командную строку с ключами (рис. 3.3).



```
#!/bin/bash
iflag=0; oflag=0; pflag=0; cflag=0; nflag=0; #Инициализация переченных-флагов, присваиваем
им 0
while getopts i:op:cn optletter          #Анализируем командную строку на наличие опций
do case $optletter in                    #Если опция присутствует в строке, то
    присваиваем ей 1
        i) iflag=1; sval=$OPTARG;;
        o) oflag=1; eval=$OPTARG;;
        p) pflag=1; pval=$OPTARG;;
        C) cflag=1;;
        n) nflag=1;;
        *) echo illegal option $optletter
    esac
done
if (($pflag==0))                        #Проверка, указан ли заголовок для поиска
then echo "Заголовок не найден"
else
    if (($iflag==0))
    then echo "Файл не найден"
    else
        if ((oflag==0))
        then if (($cflag==0))
            then if (($nflag==0))
                then grep $pval $sval
                else grep -n $pval $sval
                fi
            else if (($nflag==0))
                then grep -i $pval $sval
                else grep -i -n $pval $sval
                fi
            fi
        else if (($cflag==0))
            then if (($nflag==0))
                then grep $pval $sval > $oval
                else grep -n $pval $sval > $oval
                fi
            else if (($nflag==0))
                then grep -i $pval $sval > $oval
                else grep -i -n $pval $sval > $oval
                fi
            fi
        fi
    fi
fi
```

Рис. 3.3: Пишем код

Проверяем как работает код и находится нужная строка (рис. 3.4).

```
svzhukova@svzhukova:~/lab13$chmod +x 1.sh
svzhukova@svzhukova:~/lab13$./1.sh -p "охотник" text
Файл не найден
svzhukova@svzhukova:~/lab13$./1.sh -p "охотник" -i text
каждый охотник
svzhukova@svzhukova:~/lab13$./1.sh -p "охотник" -i text -n
1:каждый охотник
svzhukova@svzhukova:~/lab13$./1.sh -p "Охотник" -i text -n -C
1:каждый охотник
svzhukova@svzhukova:~/lab13$touch
```

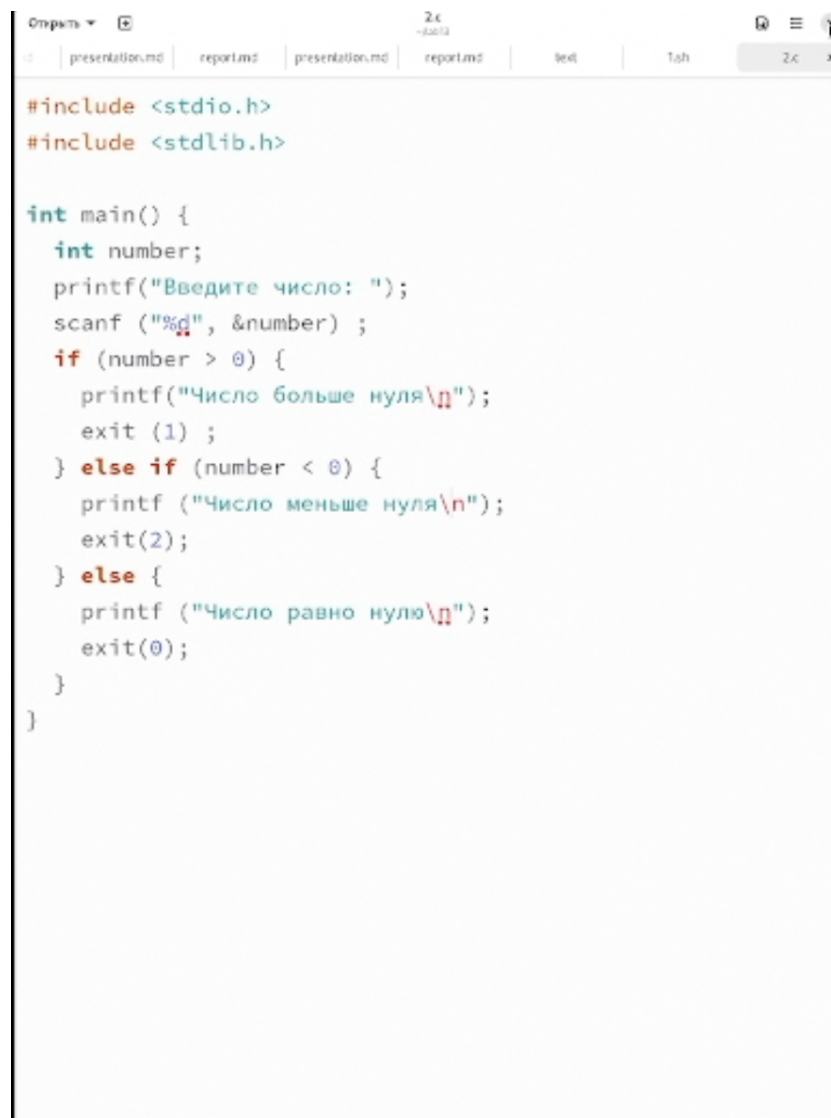
Рис. 3.4: Проверяем

Создадим файлы с расширениями sh и c (рис. 3.5).

```
svzhukova@svzhukova:~/lab13$touch 2.c
svzhukova@svzhukova:~/lab13$
```

Рис. 3.5: Создаем файлы

Пишем код, который вводит число и определяет, является ли оно больше нуля, меньше нуля или равно нулю. (рис. 3.6).

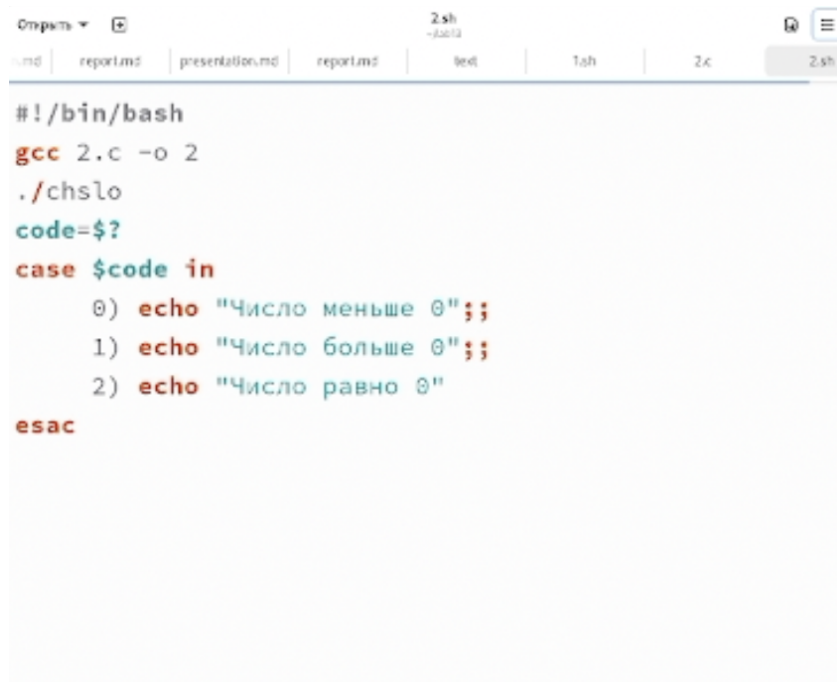
A screenshot of a code editor window. The title bar shows 'Открыть' (Open) and a file icon. Below the title bar, there are tabs for 'presentation.md', 'report.md', 'presentation.md', 'report.md', 'test', and 'Tab'. The main area displays C code with syntax highlighting. The code includes headers for stdio.h and stdlib.h, defines a main function, declares an integer variable 'number', prompts the user for input, reads the input, and uses conditional logic to check if the number is greater than, less than, or equal to zero, printing messages and exiting with status codes 1, 2, or 0 respectively.

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int number;
    printf("Введите число: ");
    scanf ("%d", &number) ;
    if (number > 0) {
        printf("Число больше нуля\n");
        exit (1) ;
    } else if (number < 0) {
        printf ("Число меньше нуля\n");
        exit(2);
    } else {
        printf ("Число равно нулю\n");
        exit(0);
    }
}
```

Рис. 3.6: Пишем код

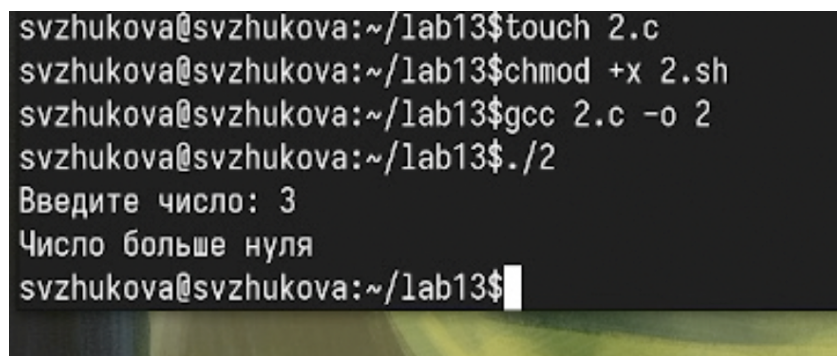
Затем программа завершается с помощью функции `exit(n)`, передавая информацию о коде завершения в оболочку (рис. 3.7).

A screenshot of a code editor window. The top bar shows a file explorer with tabs for 'report.md', 'presentation.md', 'report.md', 'test', '1.sh', '2.c', and '2.sh'. The main editor area contains a shell script in Bash. The script starts with a shebang, then compiles '2.c' to '2', runs './chsl0', and sets 'code=\$?'. It then uses a 'case' statement to check if 'code' is 0, 1, or 2, and prints Russian messages accordingly. The script ends with 'esac'.

```
#!/bin/bash
gcc 2.c -o 2
./chsl0
code=$?
case $code in
  0) echo "Число меньше 0" ;;
  1) echo "Число больше 0" ;;
  2) echo "Число равно 0"
esac
```

Рис. 3.7: Пишем код

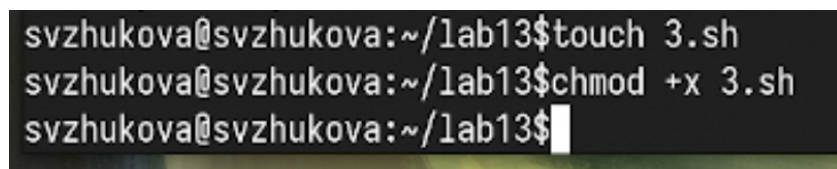
Проверяем как работает код (рис. 3.8).

A screenshot of a terminal window. The user is in the directory ~/lab13. They create a file '2.c', make '2.sh' executable, compile '2.c' to '2', and run './2'. The prompt 'Введите число:' is shown, followed by the input '3'. The script outputs 'Число больше нуля'.

```
svzhukova@svzhukova:~/lab13$ touch 2.c
svzhukova@svzhukova:~/lab13$ chmod +x 2.sh
svzhukova@svzhukova:~/lab13$ gcc 2.c -o 2
svzhukova@svzhukova:~/lab13$ ./2
Введите число: 3
Число больше нуля
svzhukova@svzhukova:~/lab13$
```

Рис. 3.8: Проверяем

Создадим файл с расширением sh и присвоим права для запуска кода (рис. 3.9).

A screenshot of a terminal window. The user is in the directory ~/lab13. They create a file '3.sh' and make it executable.

```
svzhukova@svzhukova:~/lab13$ touch 3.sh
svzhukova@svzhukova:~/lab13$ chmod +x 3.sh
svzhukova@svzhukova:~/lab13$
```

Рис. 3.9: Создаем файл

Пишем код, создающий указанное число файлов, пронумерованных последовательно от 1 до n (рис. 3.10).

```
#!/bin/bash

if [ -z "$1" ]; then
    echo "Использование: $0 <количество_файлов> [-d для удаления]"
    exit 1
fi

num_files=$1
file_prefix="file"
file_suffix=".tmp"

delete_option=false
if [ "$2" == "-d" ]; then
    delete_option=true
fi

create_files() {
    for ((i=1; i<=num_files; i++)); do
        file_name="${file_prefix}${i}${file_suffix}"
        touch "$file_name"
        echo "Создан файл: $file_name"
    done
}

delete_files() {
    for ((i=1; i<=num_files; i++)); do
        file_name="${file_prefix}${i}${file_suffix}"
        if [ -f "$file_name" ]; then
            rm -f "$file_name"
        fi
    done
}

if [ "$delete_option" = true ]; then
    delete_files
else
    create_files
fi

exit 0
```

Рис. 3.10: Пишем код

Запускаем код и создаем 5 файлов (рис. 3.11).

```
svzhukova@svzhukova:~/lab13$ ./3.sh 5
Создан файл: file1.tmp
Создан файл: file2.tmp
Создан файл: file3.tmp
Создан файл: file4.tmp
Создан файл: file5.tmp
```

Рис. 3.11: Запускаем

Проверяем как работает код (рис. 3.12).

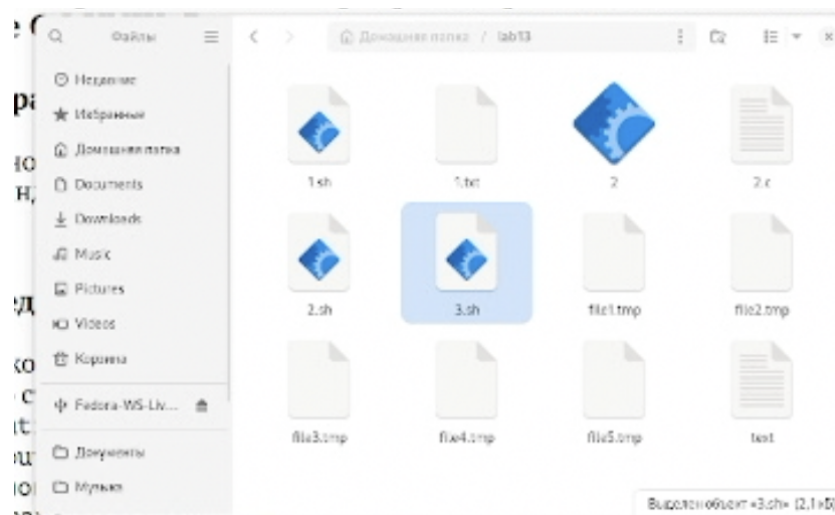


Рис. 3.12: Проверяем

Запускаем код и удаляем 5 файлов (рис. 3.13).

```
svzhukova@svzhukova:~/lab13$ ./3.sh 5 -d
Удален файл: file1.tmp
Удален файл: file2.tmp
Удален файл: file3.tmp
Удален файл: file4.tmp
Удален файл: file5.tmp
svzhukova@svzhukova:~/lab13$
```

Рис. 3.13: Запускаем

Проверяем, что файлы удалились (рис. 3.14).

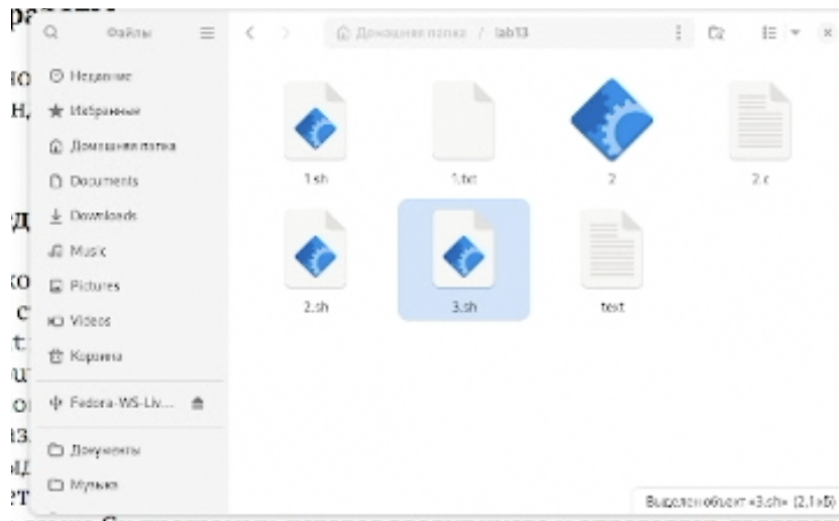


Рис. 3.14: Проверяем

Создадим файл с расширением sh и присвоим права для запуска кода (рис. 3.15).

```
svzhukova@svzhukova:~/lab13$ touch 4.sh
svzhukova@svzhukova:~/lab13$ chmod +x 4.sh
svzhukova@svzhukova:~/lab13$
```

Рис. 3.15: Создаем файл

Пишем код, который с помощью команды tar запаковывает в архив все файлы в указанной директории. (рис. 3.16).



```
#!/bin/bash

# Проверка, передан ли каталог в качестве аргумента
if [ -z "$1" ]; then
    echo "Использование: $0 <каталог> <имя_архива>"
    exit 1
fi

# Каталог для архивации
directory="$1"

# Имя архива
archive_name="$2"
if [ -z "$archive_name" ]; then
    archive_name="archive.tar.gz"
fi

# Проверка существования каталога
if [ ! -d "$directory" ]; then
    echo "Ошибка: Каталог '$directory' не существует."
    exit 1
fi

# Поиск файлов, измененных менее недели назад, и создание архива
find "$directory" -type f -mtime -7 -print0 | tar -czvf "$archive_name"
--null -T -

echo "Архив '$archive_name' создан."

exit 0
```

Рис. 3.16: Пишем код

Запускаем код (рис. 3.17).


```

svzhukova@svzhukova:~/lab13$ ./4.sh .
./1.txt
./text
./1.sh
./2.c~
./2.c
./2
./2.sh~
./2.sh
./3.sh~
./3.sh
./4.sh
Архив '$2' создан.
svzhukova@svzhukova:~/lab13$

```

Рис. 3.17: Запускаем

Проверяем как создался архив (рис. 3.19).

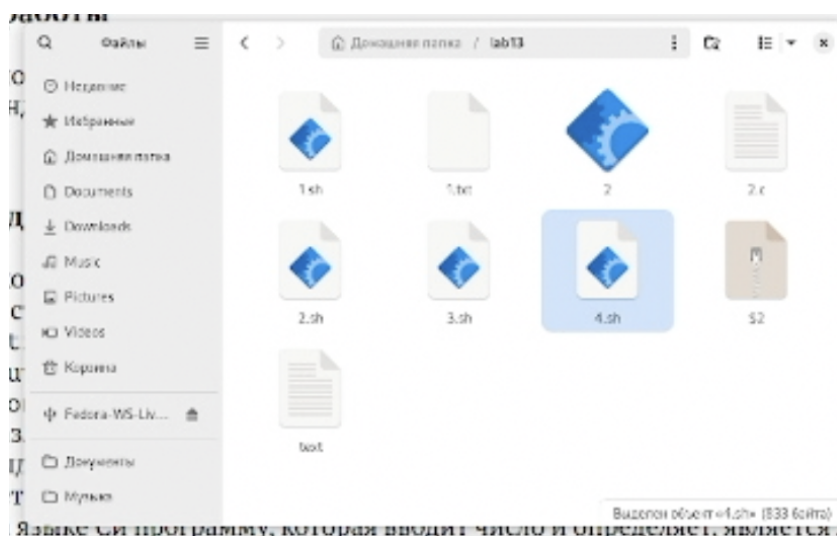


Рис. 3.18: Проверяем

Распаковываем папку (рис. 3.19).

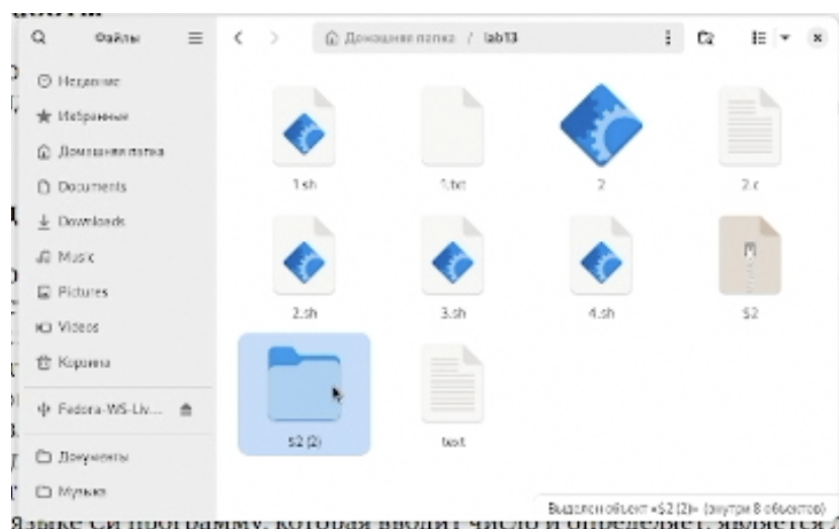


Рис. 3.19: Распаковываем

Проверяем содержимое распакованной папки (рис. 3.20).

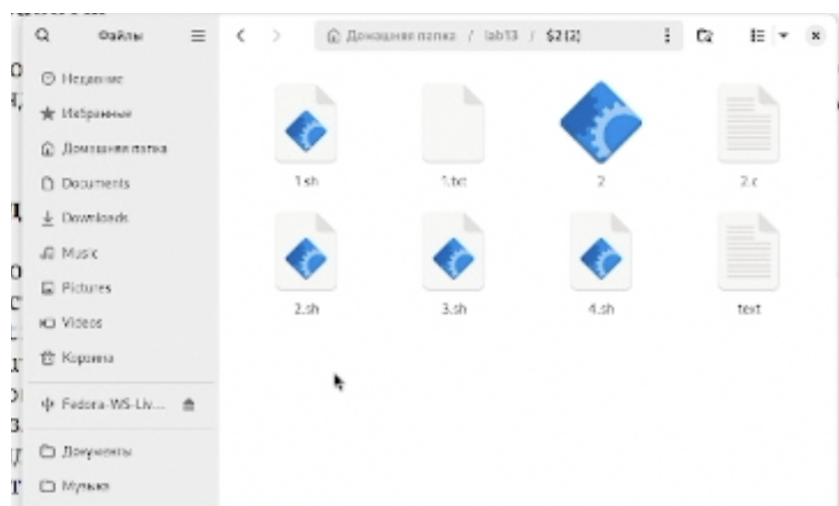


Рис. 3.20: Проверяем

4 Выводы

Мы изучили основы программирования в оболочке ОС UNIX, научились писать более сложные командные файлы с использованием логических управляющих конструкций и циклов.